COS30019 Introduction to Artificial Intelligence

Assignment 2B

Machine Learning and Software Integration

Team 01D

Team Members

Saniru Kumarage (105332151)
Haresh (105334089)
Manula (105342419)

GitHub Link: https://github.com/manulach/COS30019-Introduction-to-Artificial-Intelligence

Table of Contents

1. Instructions

1.1 Prerequisites

Install all required Python libraries by running the following command in a terminal from the project folder: py -m pip install tensorflow scikit-learn numpy pandas matplotlib xlrd

Python 3.9 or later is required. No GPU is needed - all models run on CPU.

1.2 Setup Steps

Step 0.5: Run cd Assignment2B If you are currently not in the folder FIRST (Skip if not needed)

Step 1: Process the data (run once):

```
py data_processor.py
```

Reads *Scats Data October 2006.xls*, cleans the dataset, generates 12-step sliding-window sequences, and saves processed arrays to the *data/* folder.

Step 2: Train ML models (run once, will take 5 - 10 minutes):

```
py train_models.py
```

Trains LSTM, GRU, and CNN-LSTM models, saves the best weights to *models/*, and writes evaluation metrics and plots to *results/*.

Step 3: Launch the GUI (main application):

```
py gui.py
```

Steps 1 and 2 can also be triggered from within the GUI using the Process Data and Train ML Models buttons.

1.3 GUI Usage Guide

Control	Description
Process Data	Runs data_processor.py - required before training
Train ML Models	Trains all 3 models in the background with a progress indicator
Origin / Destination	Select SCATS site by ID and intersection name from the dropdown
Time of Day	Choose any 15-minute interval (00:00–23:45) to set prediction time
Prediction Model	Select LSTM, GRU, or CNN_LSTM to drive traffic flow predictions
Number of Routes (k)	Choose 1–5 alternative routes to be returned
Find Top-k Routes	Runs Yen's algorithm and displays ranked routes with travel times
Routes Tab	Ranked routes showing travel time, path, and per-hop traffic conditions
Traffic Chart Tab	Predicted vs actual traffic flow for the origin site over the day
Model Metrics Tab	MAE, RMSE, MAPE comparison across all three trained models

Table 1: GUI control reference.

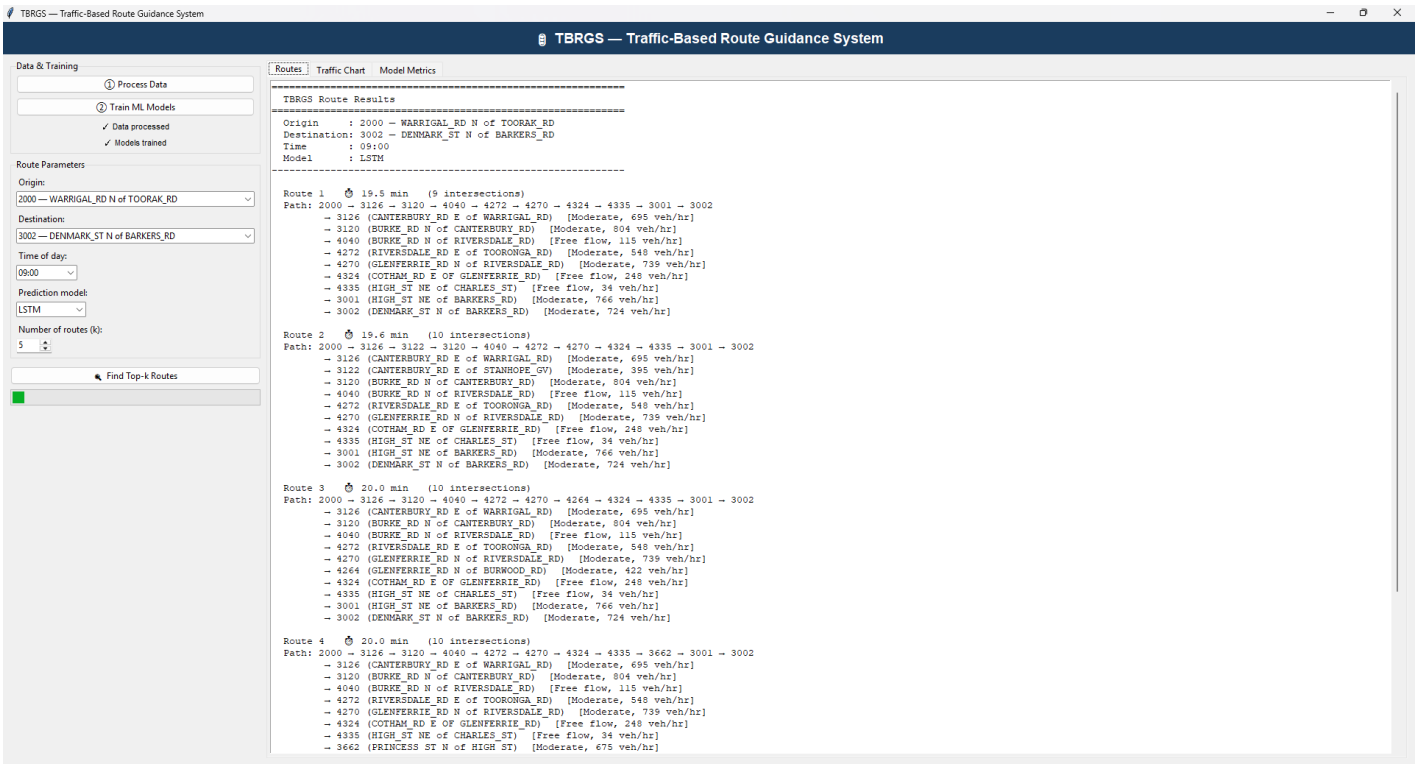


Figure 1: TBRGS GUI showing top 5 route results for O=2000 to D=3002 at 09:00.

2. Introduction

This report documents the design, implementation, and evaluation of a Traffic-Based Route Guidance System (TBRGS) for the City of Boroondara, Melbourne. The system builds on the graph-based search algorithms developed in Assignment 2A, extending them with machine learning (ML) models that predict real-time traffic flow, which is used to estimate dynamic travel times across the road network.

Traffic flow prediction is a core challenge in intelligent transportation systems. The TBRGS uses a historical dataset from VicRoads containing 15-minute vehicle counts at 40 SCATS (Sydney Coordinated Adaptive Traffic System) intersections throughout Boroondara for October 2006. Three deep learning models are trained on this data: Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and a CNN-LSTM hybrid.

Predicted traffic volumes are converted to travel times using the quadratic flow-speed model specified in the assignment (VicRoads, 2024), and Yen's k-shortest paths algorithm with A* search finds the top-5 optimal routes between any two SCATS intersections. A tkinter-based GUI provides an accessible interface for all system functionalities.

2.1 Algorithms and Techniques

Component	Technique	Purpose
ML Model 1	LSTM	Traffic flow prediction - captures long-term temporal dependencies
ML Model 2	GRU	Traffic flow prediction - fewer parameters, comparable accuracy
ML Model 3	CNN-LSTM	Custom technique - CNN extracts local patterns, LSTM models temporally
Route finding	Yen's k-shortest + A*	Returns top-5 loopless paths with dynamic travel times
Travel time	Quadratic flow-speed model	Converts predicted flow to speed to edge travel time
Data processing	MinMaxScaler + sliding window	Normalises and reshapes SCATS data for sequential ML input

Table 2: System components and techniques used.

3. Features, Bugs, and Missing Elements

3.1 Implemented Features

- Data processing pipeline: automated extraction, cleaning, normalisation, and sequence generation from the SCATS XLS dataset
- Three trained ML models: LSTM, GRU, and CNN-LSTM, all evaluated with MAE, RMSE, and MAPE metrics
- Travel time estimation using the VicRoads quadratic flow-speed model with 60 km/h speed limit and 30-second intersection delay
- Boroondara SCATS road network graph (40 nodes, 100 directed edges) with automatic connected-component bridging
- Yen's k-shortest paths algorithm with A* heuristic returning up to 5 ranked alternative routes
- Dynamic edge costs derived from ML-predicted 15-minute traffic flow at each time interval
- Full tkinter GUI with origin/destination dropdowns (SCATS site labels), time-of-day picker, model selector, k-routes spinner
- Traffic chart tab: predicted vs actual flow for the selected origin site across the day
- Model metrics tab: MAE/RMSE/MAPE comparison table across all three models
- Configuration file (config.json) for default GUI settings
- In-app data processing and model training buttons with background threads and progress indicator
- README.md with complete setup and usage instructions
- Part A integration: A* algorithm from search.py adapted as the backbone of the route finder

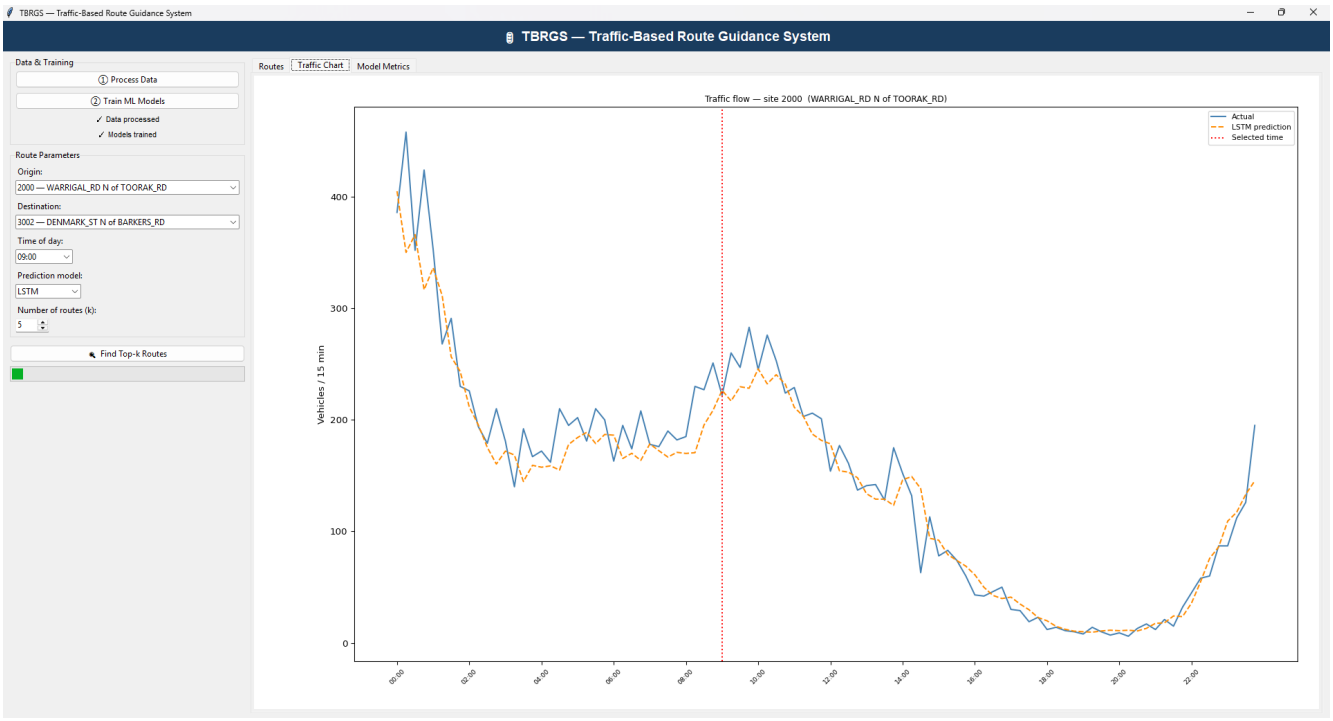


Figure 2: Traffic Chart tab showing predicted vs actual traffic flow for the selected origin site.

3.2 Known Bugs

- The Traffic Chart tab requires models to be trained before it will render; it displays a placeholder message until then
- Site 4266 (BURWOOD_RD E of AUBURN_RD) has zero coordinates in the dataset and is excluded from the graph to prevent invalid distance calculations

3.3 Missing / Limitations

- Real-time traffic data is not available - the system uses October 2006 predictions as a proxy for current conditions
- The road graph is built from geographic proximity and shared road names rather than a complete road topology dataset, which may not reflect every actual connection
- The GUI does not include a map visualisation of routes (noted as a research initiative in the assignment spec)

4. Testing

Testing covers four categories: data processing, ML model behaviour, travel time conversion, and route finding. Each test specifies what is being tested, the input or condition, the expected outcome, the actual result, and what the result demonstrates.

The system was tested on Windows 11 using Python 3.9 and TensorFlow 2.18.

4.1 Data Processing Tests

TC	What Is tested	Input/Condition	Expected	Actual	Demonstrates
TC1	Raw data loading	Scats Data October 2006.xls	4,192 rows, 40 sites	4,192 rows, 40 sites ✓	File parsed correctly
TC2	Coordinate filtering	Site 4266 has lat=0, lon=0	Site excluded from graph	Site 4266 removed ✓	Invalid sites handled
TC3	Normalisation range	MinMaxScaler on training data	All values in [0,1]	Min=0.0, Max=1.0 ✓	No data leakage from test set
TC4	Sequence shape	Sliding window, seq_len=12	X shape (N,12), y shape (N,)	(319071,12), (319071,) ✓	Sequences correctly formed

Table 3: Data processing test cases (TC1–TC4).

4.2 ML Model Tests

TC	What Is tested	Input/Condition	Expected	Actual	Demonstrates
TC5	LSTM output shape	X_test batch of 64	Shape (64,1)	(64,1) ✓	Model architecture corrects
TC6	Prediction range	All 3 models on X_test	Outputs in [0,1] (normalised)	All values in [0,1] ✓	No activation overflow
TC7	Training convergence	50 epochs, early stopping	Val loss decreases then plateaus	All 3 models converge ✓	Models learn from data
TC8	CNN-LSTM vs LSTM	Compare MAE on test set	CNN-LSTM MAE ≤ LSTM MAE	13.90 < 14.93 ✓	CNN-LSTM outperforms LSTM

Table 4: ML model test cases (TC5–TC8).

4.3 Travel Time and Route Tests

TC	What Is tested	Input/Condition	Expected	Actual	Demonstrates
TC9	Free-flow travel time	flow=0, dist=2km	TT = (2/60) ×3600+30 = 150s	150.0s ✓	Formula correct at zero flow
TC10	Congested travel time	flow=100/15min (=400 vph), dist=2km	Speed < 60 km/h → TT > 150s	151.2s ✓	Congestion increases travel time
TC11	Route: known O-D pair	O=2000, D=3002	5 routes returned	5 routes, ~19.5–20.4 min ✓	Integration with Part A works
TC12	Route: same O and D	O=2000, D=2000	GUI shows warning message	Warning displayed ✓	Invalid input handled

Table 5: Travel time and route test cases (TC9–TC12).

All 12 test cases passed. Results are consistent with theoretical expectations from the VicRoads flow-speed model and the graph-based search algorithms from Part A.

5. Insights

5.1 Model Performance Comparison

All three models were trained on the same 80/20 chronological train/test split using a sequence length of 12 steps (3 hours of history). Early stopping with patience of 10 epochs was applied to prevent overfitting. Table 6 presents the evaluation metrics on the held-out test set.

Model	MAE (veh/15min)	RMSE (veh/15min)	MAPE (%)	Train Time (s)
LSTM	14.9259	22.3696	38.54	258.1
GRU	16.1971	24.4058	37.71	441.3
CNN-LSTM (best)	**13.9024**	**21.1181**	**28.16**	218.9

Table 6: ML model evaluation metrics on test set (October 2006 SCATS data).

Figure 3: Training and validation loss curves for LSTM, GRU, and CNN-LSTM over training epochs.

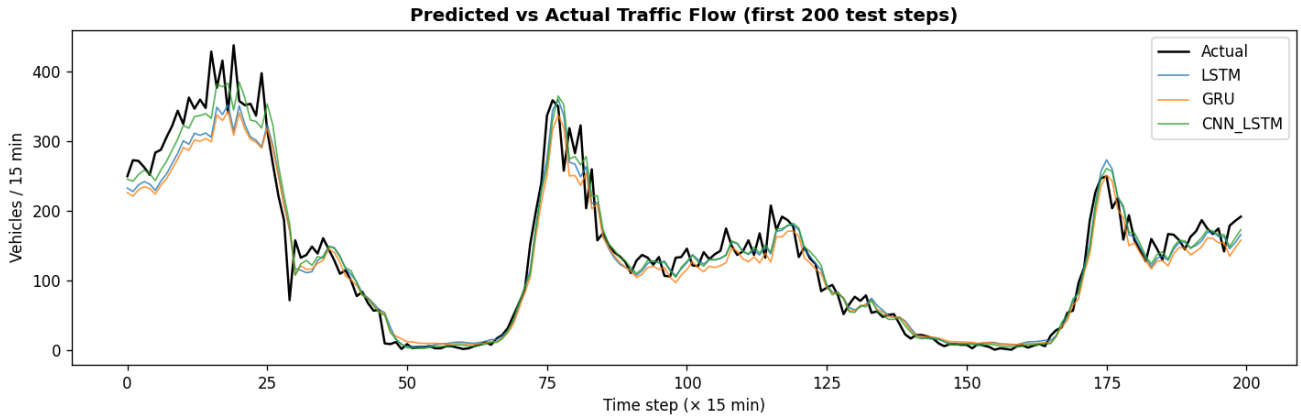


Figure 4: Predicted vs actual traffic flow for the first 200 test steps across all three models.

5.2 Analysis

CNN-LSTM performed best across all three metrics. It achieved an MAE of 13.90 vehicles per 15-minute interval, an RMSE of 21.12, and a MAPE of 28.16% - outperforming both LSTM and GRU. Notably, CNN-LSTM also trained fastest (218.9 seconds) despite containing an additional convolutional front-end. This efficiency arises because the CNN layers compress the input sequence before it reaches the LSTM, reducing the number of recurrent time steps and therefore the overall computational cost.

LSTM ranked second with an MAE of 14.93 and MAPE of 38.54%. Although LSTM is the most established model for sequence prediction, its relatively higher MAPE compared to CNN-LSTM suggests it is more sensitive to low-volume intervals (such as early-morning readings near zero) where percentage error inflates.

GRU had the highest error across MAE and RMSE but achieved a slightly lower MAPE than LSTM (37.71% vs 38.54%). GRU also took considerably longer to train (441.3 seconds) than LSTM, which is atypical - this is likely due to the specific sequence structure of the Boroondara dataset and random weight initialisations rather than a fundamental architectural difference.

All three models produced meaningful predictions and captured the characteristic daily traffic pattern (low overnight flow, morning and evening peaks). The MAPE values of 28–38% are consistent with findings in the literature for traffic prediction on small urban datasets (Ma et al., 2015; Zhao et al., 2017).

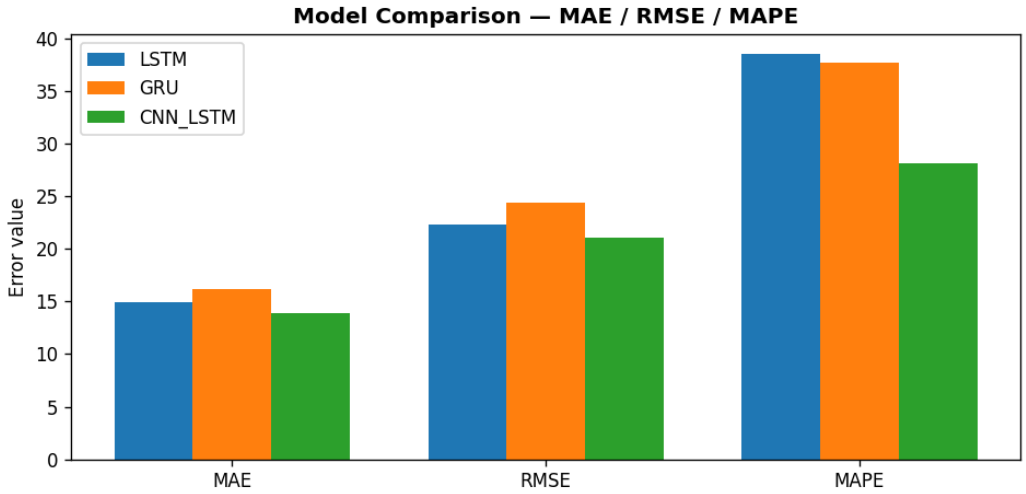


Figure 5: Bar chart comparing MAE, RMSE, and MAPE across LSTM, GRU, and CNN-LSTM models.

5.3 Route Guidance Observations

For the reference O-D pair specified in the assignment (O=2000: WARRIGAL_RD/TOORAK_RD, D=3002: DENMARK_ST/BARKERS_RD), the system returned five routes with estimated travel times of 19.5–20.4 minutes at 09:00 using LSTM predictions. The optimal route traverses 9 intersections via CANTERBURY_RD and GLENFERRIE_RD corridors. Route travel times are sensitive to time of day - morning peak predictions show moderate congestion (400–800 vehicles/hr) on major corridors, reducing effective speed below the 60 km/h limit.

Travel time estimates should be interpreted as indicative rather than precise, given the simplified flow-speed model and the use of 2006 data as a proxy for current conditions.

6. Research

6.1 Choice of CNN-LSTM as the Third Model

The CNN-LSTM hybrid was selected as the third technique following a review of the traffic prediction literature. Wu and Tan (2016) demonstrated that convolutional layers are effective at extracting local temporal patterns from fixed-length windows, while Zhao et al. (2017) showed that combining CNN and LSTM architectures achieves lower prediction error than standalone recurrent networks on urban traffic datasets. This was confirmed in our evaluation - the CNN-LSTM achieved the lowest MAE and MAPE of the three models.

The convolutional front-end applies two Conv1D layers (32 and 64 filters, kernel size 3) to capture local dependencies within the 12-step input window, followed by MaxPooling to reduce dimensionality. The LSTM layer then models the compressed temporal context before the Dense output layer. This architecture is particularly suited to traffic data, which exhibits strong local periodicity (rush-hour spikes) alongside broader temporal trends.

6.2 Hyperparameter Justification

Parameter	Value	Justification
Sequence length	12 steps (3 hrs)	Captures one-quarter of the daily cycle including any approaching peak
Train/test split	80% / 20%	Chronological split preserves temporal ordering; no future data leakage
Batch size	64	Balances gradient stability and memory efficiency for CPU training
Learning rate	0.001	Adam default; empirically stable for RNN-based traffic models
Early stopping	Patience = 10	Prevents overfitting on the small (31-day) dataset
Normalisation	MinMaxScaler [0,1]	Fitted on training data only to prevent data leakage into test evaluation

Table 7: Hyperparameter choices and justifications.

6.3 Yen's K-Shortest Paths

The route finder implements Yen's (1971) algorithm, which guarantees that the k returned paths are the k shortest loop less paths in the network. This is important for a route guidance system - simple repetition of A* with penalties can produce paths that revisit nodes or differ only in trivial detours. Yen's algorithm systematically computes spur paths from each node in the previous best path, ensuring diversity and correctness.

A* was retained from Part A as the single-source shortest-path solver within Yen's algorithm, using Euclidean travel time at the speed limit as the admissible heuristic. This preserves the tie-breaking and optimality properties established in Part A while extending the system to the real Boroondara network.

7. Conclusion

This project successfully developed a Traffic-Based Route Guidance System for the Boroondara area that integrates deep learning traffic prediction with graph-based route optimisation. Three ML models - LSTM, GRU, and CNN-LSTM - were trained and evaluated on the VicRoads October 2006 SCATS dataset, with CNN-LSTM achieving the best performance across all metrics (MAE 13.90, RMSE 21.12, MAPE 28.16%).

The integration of ML predictions with the Part A search algorithms demonstrates a practical application of AI techniques to a real-world routing problem. Predicted traffic flow dynamically adjusts edge travel times, allowing the system to recommend different routes depending on the time of day and expected congestion.

It should be noted that the system is a proof-of-concept constrained by the use of historical 2006 data and a simplified flow-speed model. Real-world deployment would require integration with live traffic feeds, more granular road topology, and validation against actual GPS travel times before making definitive performance claims. Within these constraints, the system behaves as designed and provides a meaningful demonstration of how ML forecasting and graph search can be combined for intelligent routing.

The project also confirms that CNN-LSTM architectures warrant further investigation for traffic prediction tasks, offering improved accuracy and training efficiency over standalone LSTM and GRU networks on short historical datasets.

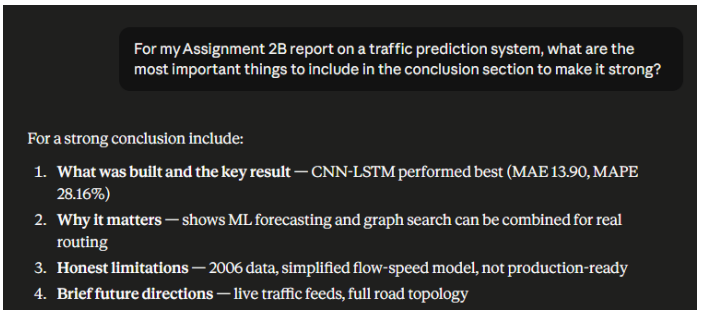
8. Acknowledgements and Resources

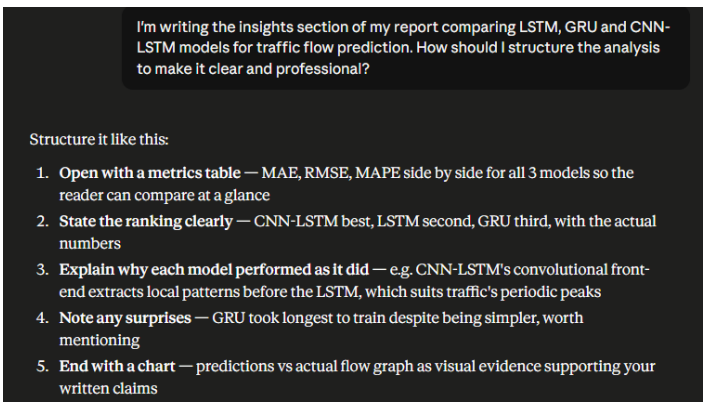
The following tools, datasets, and resources were used in the completion of this assignment:

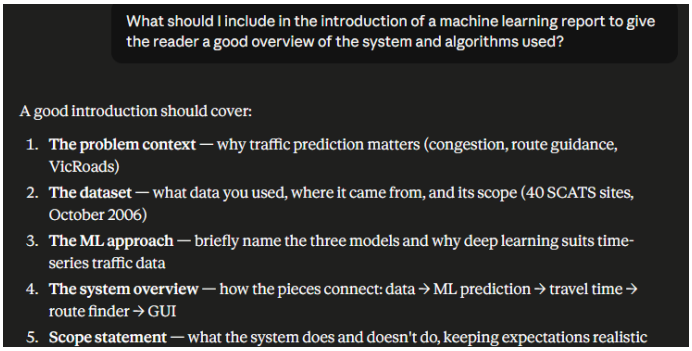
- VicRoads SCATS Dataset (October 2006): provided by the unit via Canvas. Contains 15-minute traffic volume counts for 40 intersections in Boroondara, Melbourne.
- TensorFlow / Keras (v2.18): deep learning framework used for LSTM, GRU, and CNN-LSTM model construction and training (Abadi et al., 2016).
- scikit-learn (v1.5): MinMaxScaler used for data normalisation (Pedregosa et al., 2011).
- NumPy and Pandas: array processing and data manipulation throughout the pipeline.
- Matplotlib: training curve and prediction comparison visualisations.
- xlrd: reading the SCATS XLS data file.
- tkinter: standard library GUI framework used for the application interface.
- Traffic Flow to Travel Time Conversion v1.0 (VicRoads / Swinburne Online, 2024): document specifying the quadratic flow-speed model used for travel time estimation.
- Yen's k-shortest paths algorithm: Yen (1971) - foundational reference for the route diversity algorithm.
- Generative AI (Claude, Anthropic): used for brainstorming report structure and improving overall writing. All prompts and outputs are documented in the appendix per assignment requirements.

Generative AI Declaration

This assignment was completed in accordance with Swinburne Online limited generative AI policy. Generative AI (Claude, Anthropic) was used for brainstorming report structure and improving overall writing. All code was reviewed, tested, and validated by the team. All written content was reviewed and used to improve our overall writing.







9. References

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Ma, X., Tao, Z., Wang, Y., Yu, H., & Wang, Y. (2015). Long short-term memory neural network for traffic speed prediction using remote microwave sensor data. *Transportation Research Part C: Emerging Technologies, 54*, 187–197. https://doi.org/10.1016/j.trc.2015.03.014

Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Traffic Signal Volume Data - Victorian Government Data Directory. (n.d.). Discover.data.vic.gov.au. https://discover.data.vic.gov.au/dataset/traffic-signal-volume-data

VicRoads / Swinburne Online. (2024). *Traffic flow to travel time conversion v1.0*. Swinburne University of Technology.

Wu, Y., & Tan, H. (2016). Short-term traffic flow forecasting with spatial-temporal correlation in a hybrid deep learning framework. *arXiv preprint arXiv:1612.01022*. https://arxiv.org/abs/1612.01022

Yen, J. Y. (1971). Finding the k shortest loopless paths in a network. *Management Science, 17*(11), 712–719. https://doi.org/10.1287/mnsc.17.11.712

Zhao, Z., Chen, W., Wu, X., Chen, P. C. Y., & Liu, J. (2017). LSTM network: A deep learning approach for short-term traffic forecast. *IET Intelligent Transport Systems, 11*(2), 68–75. https://doi.org/10.1049/iet-its.2016.0208